

BOARD OF INTERMEDIATE EDUCATION KARACHI

HIGHER SECONDARY · COMPUTER SCIENCE

Class XI

Lecture notes Computer Science

Sindh Curriculum 2024 · National Curriculum of Pakistan 2022–23

Original lecture notes for classroom and self-study

Session 2026–27

How to use these lectures

These lectures are written for Higher Secondary students taking Computer Science with the Board of Intermediate Education Karachi. They follow the Sindh Curriculum of Computer Science 2024, which is aligned with the National Curriculum of Pakistan 2022–23 and with the Sindh Textbook Board books for the 2026–27 session. Each lecture is one teaching block: the idea, a worked example, and a short checkpoint in the style of the HSSC paper. Python programs that are complete scripts show the output produced by running them on Python 3.

What each lecture contains

- **Learning outcomes** match the student learning outcomes of the National Curriculum of Pakistan 2022–23 for this grade.
- **Explanations and worked examples** are written so that a student can revise a period without a second textbook open.
- **Python programs** that are complete scripts include the output produced by running them.
- **Checkpoint** questions follow the HSSC pattern: one-mark choices and three-mark short answers, with a key underneath.

A note on the source of the course

Karachi intermediate colleges teach the Sindh Textbook Board books. For 2026–27 those books follow the Sindh Curriculum of Computer Science 2024, which is aligned with the National Curriculum of Pakistan 2022–23. Class XI and Class XII are organised by the same domains: Computer Systems, Computational Thinking and Algorithms, Programming Fundamentals, Data and Analysis, Applications and Impacts of Computing, Digital Literacy, and Entrepreneurship in the Digital Age. Programming in this course is Python, not the older C++ syllabus.

REMEMBER

These notes are an original teaching set. They are not an official BIEK paper, not a past paper, and not a copy of the Sindh Textbook Board book. Use the textbook alongside them for exercises set by your teacher.

Units in this booklet

Unit	Title	Emphasis in the course
Unit 1	Computer Systems	About 30 periods
Unit 2	Computational Thinking and Algorithms	About 20 periods
Unit 3	Programming Fundamentals	About 40 periods
Unit 4	Data and Analysis	About 25 periods
Unit 5	Applications and Impacts of Computing	About 30 periods
Unit 6	Digital Literacy and Prototyping	About 20 periods

Contents

Unit 1 Computer Systems	5
Lecture 1.1 Analog signals and digital data	5
Lecture 1.2 Logic gates and Boolean operations	6
Lecture 1.3 Truth tables, logic circuits, and Karnaugh maps	8
Lecture 1.4 The software development life cycle	10
Lecture 1.5 Networks, topologies, and protocol stacks	12
Lecture 1.6 Cybersecurity and encryption	14
Unit 2 Computational Thinking and Algorithms	15
Lecture 2.1 Decomposition, patterns, and abstraction	16
Lecture 2.2 Algorithms, pseudocode, and flowcharts	17
Lecture 2.3 Linear search and binary search	18
Lecture 2.4 Bubble sort and insertion sort	20
Unit 3 Programming Fundamentals	21
Lecture 3.1 Programs, Python, and the editor	21
Lecture 3.2 Values, types, operators, and input	23
Lecture 3.3 Decisions and repetition	24
Lecture 3.4 Lists	26
Lecture 3.5 Functions	27
Lecture 3.6 Libraries and turtle graphics	28
Lecture 3.7 Finding and fixing mistakes	30
Unit 4 Data and Analysis	31
Lecture 4.1 From a flat file to a database	31
Lecture 4.2 Relationships and entity-relationship diagrams	32
Lecture 4.3 Summaries and charts	34
Lecture 4.4 Straight-line models, correlation, and experiments	35
Unit 5 Applications and Impacts of Computing	36
Lecture 5.1 Cloud, the Internet of Things, and blockchains	36
Lecture 5.2 Artificial intelligence and the people it affects	38
Lecture 5.3 Information, connectivity, and access	39
Unit 6 Digital Literacy and Prototyping	40

Lecture 6.1 Searching, collecting, and presenting	41
Lecture 6.2 Prototyping an idea	42

Unit 1 Computer Systems

A computer system is hardware, software, data, and the people who use them, joined by rules for how information moves. This unit starts with the difference between a smooth analog signal and a digital value, then builds the logic that processors use. It continues with the way a software project is planned, the shapes of a network, and the protection of data while it travels.

OUTCOMES

By the end of this unit you should be able to: Distinguish analog and digital representation and build truth tables for the common logic gates. Simplify a small Boolean expression and relate it to a logic diagram. Describe the software development life cycle and choose Waterfall or Agile for a given case. Compare network topologies and explain why encryption is used.

Lecture 1.1 Analog signals and digital data

OUTCOMES

Explain analog and digital representation with an everyday example. State what a bit is and why computers store information as bits.

Two ways of describing the world

An analog quantity can take any value inside a range. The height of mercury in a thermometer, the loudness of a voice, and the position of the minute hand are analog; between two readings there is always another reading. A digital quantity jumps from one allowed value to the next. A light switch is digital in the everyday sense. Inside a computer the allowed values are only 0 and 1.

A microphone produces a voltage that follows the shape of a sound wave. That voltage is analog. To store the sound, an analog-to-digital converter measures the voltage many times each second and writes each measurement as a binary number. Playback reverses the process. The measurement is called a sample. More samples, and more bits per sample, keep more of the original shape, and they also use more storage.

Bits, patterns, and why 0 and 1 are enough

A **bit** is one binary digit, either 0 or 1. With one bit there are 2 patterns. With two bits there are 4 patterns: 00, 01, 10, and 11. With n bits there are 2 to the power n patterns. Eight bits make one **byte**, which can represent 256 different patterns. Those patterns can stand for a number, a character, a colour, or an instruction. The meaning comes from the code we agreed to use, not from the bits themselves.

Bits	Number of patterns	A familiar use
1	2	A single yes or no flag
4	16	One hexadecimal digit
8	256	One byte, often one character in older codes
16	65,536	A Unicode code unit in UTF-16

Figure. How the number of patterns grows. You do not need to memorise Unicode; you need the doubling rule.

Digital representation is used because a 0 and a 1 can be told apart even when the electrical signal is slightly noisy. A voltage near 0 V is read as 0. A voltage near the supply is read as 1. Small interference rarely flips the reading. An analog voltage has no such gap, so noise changes the value itself.

Hardware, software, and the block picture

Every general-purpose computer still follows the same block picture. An **input** device brings data in. A **processor** follows instructions. **Memory** holds the instructions and the data while the work is happening. **Storage** keeps data after the power is off. An **output** device presents the result. **Software** is the set of instructions. System software, such

as the operating system and device drivers, runs the machine. Application software, such as a browser or a Python editor, does a job the user cares about.

- **Keyboard, mouse, microphone, scanner, touch screen:** input.
- **Monitor, printer, speaker, projector:** output.
- **RAM:** primary memory, fast, and lost when power is removed.
- **SSD, hard disk, USB flash drive:** secondary storage, kept without power.
- **NIC, switch, router, modem:** communication devices.

DEFINITION

A computer system is the combination of hardware, software, data, and procedures that turns input into output under the control of a stored program.

EXAM TIP

If a question says 'give an example', name a device and say which block it belongs to. 'Mouse' alone is weaker than 'a mouse is an input device that reports movement and clicks'.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which pair is analog data followed by digital data?
 - A. A switch position, then a spoken vowel
 - B. Room temperature on a glass thermometer, then that temperature stored as a binary number
 - C. A bit, then a sound wave
 - D. A JPEG file, then the light entering a camera lens
2. Why can a computer reconstruct a song from bits if the original sound was a smooth wave? Give two points.
3. A code uses 3 bits per symbol. How many different symbols can it represent? Show the reasoning.

ANSWERS

1. B. The thermometer reading varies continuously. The stored binary number is a digital sample of it.
2. The wave is measured many times a second (sampling). Each measurement is stored as a binary number. On playback those numbers are turned back into voltages that approximate the original wave. More bits and more samples make the approximation closer.
3. Each bit has 2 states, so 3 bits give $2 \times 2 \times 2 = 8$ symbols, from 000 through 111.

Lecture 1.2 Logic gates and Boolean operations

OUTCOMES

Define AND, OR, NOT, NAND, and XOR and give the Boolean expression for each. Apply the basic Boolean identities, including De Morgan's laws.

Boolean values

Digital circuits treat 1 as true and 0 as false. George Boole's algebra is the language of those circuits. A Boolean variable such as A can only be 0 or 1. An expression combines variables with operations. The result is also 0 or 1.

The gates you must be able to draw

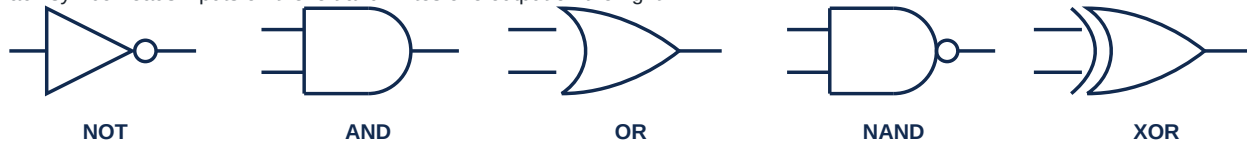
NOT has one input. The output is the opposite of the input. If A is 0, A' is 1. If A is 1, A' is 0. The dash is pronounced 'not A' or 'A bar'.

AND outputs 1 only when every input is 1. For two inputs the expression is $A \cdot B$, often written AB. Think of two switches in series: the lamp is on only when both switches are closed.

OR outputs 1 when at least one input is 1. The expression is $A + B$. This plus sign is not arithmetic addition. $1 + 1$ in Boolean OR is 1, not 2. Think of two switches in parallel: either switch can light the lamp.

NAND is AND followed by NOT. The output is 0 only when every input is 1. **NOR** is OR followed by NOT. **XOR**, exclusive OR, outputs 1 when the inputs are different. Its expression is $A \text{ XOR } B$, which is $AB' + A'B$. XOR is the gate behind a half-adder's sum bit.

Each symbol reads inputs on the left and writes one output on the right.



The five gates used most often in this course.

Gate	Expression	Output is 1 when
NOT	A'	A is 0
AND	AB	A and B are both 1
OR	$A + B$	A is 1 or B is 1 or both are 1
NAND	$(AB)'$	at least one of A or B is 0
XOR	$A \text{ XOR } B$	A and B differ

Table. Expressions use the textbook convention: juxtaposition means AND, + means OR.

Identities worth memorising

These identities let you shorten an expression before you build the circuit. A shorter expression usually means fewer gates.

- $A + 0 = A$ and $A + 1 = 1$. $A \cdot 0 = 0$ and $A \cdot 1 = A$.
- $A + A = A$ and $A \cdot A = A$. $A + A' = 1$ and $A \cdot A' = 0$.
- $(A')' = A$.
- $A + B = B + A$ and $AB = BA$. Order does not matter.
- $A(B + C) = AB + AC$. AND distributes over OR. OR also distributes over AND: $A + BC = (A + B)(A + C)$.
- **De Morgan:** $(AB)' = A' + B'$ and $(A + B)' = A'B'$. A NAND of two inputs is the OR of their complements. A NOR is the AND of their complements.

EXAMPLE

Show that $(A + B)'$ is not the same as $A' + B'$. Let $A = 1$ and $B = 0$. Then $A + B = 1$, so $(A + B)' = 0$. But $A' + B' = 0 + 1 = 1$. The two sides differ, so they are not identical. De Morgan says the complement of an OR is the AND of the complements: $A'B' = 0 \cdot 1 = 0$, which matches.

Duality is a bookkeeping rule. If an identity is true, the identity obtained by swapping AND with OR, and swapping 0 with 1, is also true. From $A + 0 = A$ the dual is $A \cdot 1 = A$. You still have to keep the complements as they are. Duality is a way to remember a second identity, not a new fact about a particular circuit.

REMEMBER

NAND and NOR are called universal gates because any other gate can be built from NAND alone, or from NOR alone. You will not be asked to build a full set in the theory paper, but you should know the name.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which expression is the XOR of A and B?

- A. $AB + A'B'$
- B. $AB' + A'B$
- C. $(A + B)'$
- D. $A + B$

2. State both of De Morgan's laws and check $(AB)' = A' + B'$ for $A = 1, B = 1$.

3. A lamp is controlled by two switches. The lamp should light when the switches are in different positions. Which gate models this, and why is OR the wrong choice?

ANSWERS

1. B. XOR is 1 when the inputs differ: A is 1 and B is 0, or A is 0 and B is 1.
2. $(AB)' = A' + B'$ and $(A + B)' = A'B'$. When $A = 1$ and $B = 1$, $AB = 1$ so $(AB)' = 0$. $A' + B' = 0 + 0 = 0$. Both sides match.
3. XOR, because its output is 1 only when the inputs differ. OR would also light the lamp when both switches are on, which is not required.

Lecture 1.3 Truth tables, logic circuits, and Karnaugh maps

OUTCOMES

Build a truth table of up to three inputs and draw the matching logic circuit. Group cells on a 3-variable Karnaugh map and read off a simpler expression.

From an expression to a truth table

A truth table lists every input combination once. For two inputs there are 4 rows. For three inputs there are 8 rows. Count in binary so you do not skip a row: 000, 001, 010, 011, 100, 101, 110, 111. Add a column for each intermediate expression, then a column for the final output. Intermediate columns catch arithmetic mistakes.

Take $F = AB' + C'$. The intermediate columns are B' , AB' , C' , and then F .

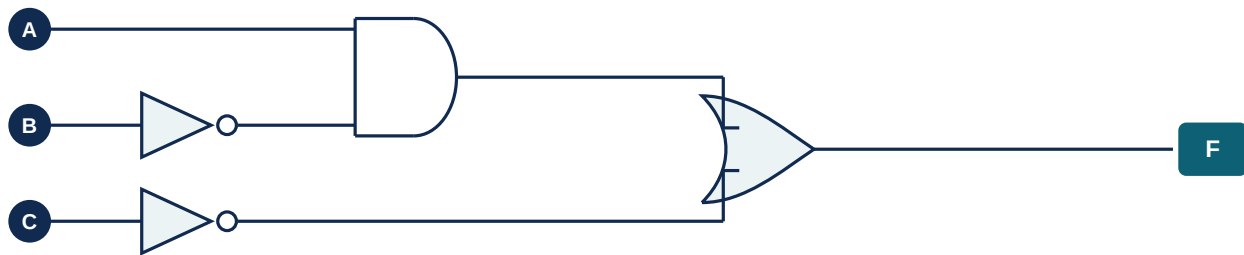
A	B	C	B'	AB'	C'	F
0	0	0	1	0	1	1
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	0	0	0
1	0	0	1	1	1	1
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	1	0	0	0	0

Table. F is 1 for minterms 0, 2, 4, 5, and 6. Check the row $A=1, B=0, C=1$: AB' is 1, so F is 1 even though C' is 0.

From an expression to a circuit

Read the expression from the inside out. Complements become NOT gates. A product becomes an AND gate. A sum becomes an OR gate. For $F = AB' + C'$ you need a NOT on B, a NOT on C, an AND of A and B', and an OR that combines the AND output with C'. Label every wire. A circuit without labels is hard to mark.

Logic circuit for $F = A \cdot B' + C'$



Worked circuit: invert B and C, AND A with B', then OR that result with C'.

Karnaugh maps

A Karnaugh map, or K-map, is a truth table redrawn so that combinations which differ by one variable sit next to each other. That neighbour rule is why the column headings are in Gray-code order: 00, 01, 11, 10, not ordinary binary counting. The cells for 00 and 10 are also neighbours because the map wraps around the edges.

Write 1 in every cell whose minterm makes F true, and 0 in the rest. Then circle groups of 1, 2, 4, or 8 cells. Groups must be rectangles, wrapping is allowed, and groups may overlap. A larger group drops more variables. Any variable that changes inside the group is removed. A variable that stays 1 is written as itself. A variable that stays 0 is written complemented.

K-map for $F = m(0, 2, 4, 5, 6)$. Column order of BC is 00, 01, 11, 10.

A	00	01	11	10	
0	1	0	0	1	Gold pair = AB' Cells with $C = 0$ are one group: C'
1	1	1	0	1	

$F = C' + AB'$

Adjacent 1s are grouped in sizes 1, 2, 4, or 8. Opposite edges of a K-map are adjacent.

In this map the four cells where $C = 0$ form one group. Inside that group A changes and B changes, so both disappear, leaving C' . The two cells in row $A = 1$ under $BC = 00$ and $BC = 01$ form a second group. C changes, so C disappears. A stays 1 and B stays 0, leaving AB' . No 1 is left outside these groups. Therefore $F = C' + AB'$. The truth table of this shorter expression is the same eight rows as the table above.

EXAM TIP

After you simplify, substitute one input row from the original table into the new expression. If the output disagrees, the grouping is wrong. This check takes less time than redrawing the map.

REMEMBER

Do not group zeros unless the question asks for the complement. Do not make a group of 3 or 6. Only powers of two are legal groups.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A 3-variable truth table has how many rows?
 - A. 3
 - B. 6
 - C. 8
 - D. 9
2. Copy the row $A=1, B=1, C=0$ of $F = AB' + C'$ and show the value of each intermediate column.
3. On a K-map, why are the headings 00, 01, 11, 10 rather than 00, 01, 10, 11?

ANSWERS

1. C. Three inputs give $2^3 = 8$ combinations.
2. $B' = 0, AB' = 1 \cdot 0 = 0, C' = 1, F = 0 + 1 = 1$.
3. Adjacent headings must differ by exactly one bit so that neighbouring cells differ by one variable. 01 and 10 differ in two bits, so they must not be neighbours. Gray-code order 00, 01, 11, 10 keeps that promise, and the first and last columns are also neighbours.

Lecture 1.4 The software development life cycle

OUTCOMES

Name the stages of the software development life cycle and the work done in each. Choose Waterfall or Agile for a case and justify the choice with feasibility.

Why a life cycle exists

A program written straight into the editor often solves the wrong problem. The **software development life cycle (SDLC)** is the set of stages a team passes through so that the finished system matches a real need, can be tested, and can be changed later. A **bug** is a fault that makes the system behave differently from its specification. Finding a bug in the design stage is cheaper than finding it after the college is already depending on the system.



Waterfall moves one way. Agile repeats design, code, and test in short cycles.

The seven names used for the life cycle in this course.

- **Planning.** What is the problem, who is affected, and is the project worth doing?
- **Requirements analysis.** What must the system do (functional requirements) and under what limits of speed, security, and law (non-functional requirements)? The written result is often a software requirements specification.
- **Design.** How will the parts fit: screens, data, and modules? Algorithms and flowcharts belong here.
- **Implementation.** Programmers write the code.
- **Testing.** The team looks for bugs. **Black-box** testing uses inputs and outputs only, as a user would. **White-box** testing looks at the code and tries to exercise paths inside it.
- **Deployment.** The system is put into real use.
- **Maintenance.** Faults are repaired, and the system is adapted when rules or hardware change.

Feasibility

Before a large build starts, the team asks whether the project is feasible. **Technical** feasibility asks whether the skills, devices, and software exist. **Economic** feasibility asks whether the benefit justifies the cost. **Operational** feasibility asks whether people can and will use the system in their actual work. **Legal** feasibility asks whether the system stays inside the law, including rules on student or patient data. A hospital records system whose development cost is above the budget fails economic feasibility even if the programmers know how to build it.

Waterfall and Agile

Waterfall finishes one stage before the next begins. It fits work whose requirements are stable and whose sequence is forced by the physical world: a college building, a road, a production line for a known car model. Changing an early decision is expensive because later stages were built on it.

Agile builds in short cycles. Each cycle delivers a small working piece, the users react, and the next cycle changes course. It fits a college website, a digital marketing campaign, or a research tool, because the right behaviour is discovered while the work is going on. Agile still analyses, designs, codes, and tests. It repeats those activities instead of doing each one only once.

Situation	Better fit	Reason
Public drainage project	Waterfall	The specification is fixed by engineering standards and the work cannot be 'released' half-dug in weekly slices.
College web application	Agile	Students and staff will change the required screens once they try a first version.
Research dashboard	Agile	The questions change as the data is understood.
Mass production of one approved car model	Waterfall	The design is frozen before the line is tooled.

Table. The method follows the stability of the requirements, not the size of the organisation.

EXAM TIP

A 'which model' question is marked on the reason. Write the reason in the same sentence as the name.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Typing a username and password and checking that the dashboard opens, without reading the source code, is an example of
 - A. white-box testing
 - B. black-box testing
 - C. economic feasibility
 - D. maintenance
2. A clinic wants new patient software. The quoted cost is higher than the allocated budget and higher than the expected savings. Which feasibility has failed, and which SDLC stage should catch this?
3. Give one project suited to Waterfall and one suited to Agile. One sentence of justification each.

ANSWERS

1. B. Only the external behaviour is observed. White-box testing would inspect the code.
2. Economic feasibility has failed. It should be examined during planning, before design and coding begin.
3. Building a classroom block suits Waterfall because the plan is approved before construction and stages depend on each other. Building the college's mobile notice app suits Agile because staff will revise the screens after trying the first release.

Lecture 1.5 Networks, topologies, and protocol stacks

OUTCOMES

Describe sender, receiver, medium, message, and protocol. Compare bus, star, ring, and mesh on reliability and the ease of adding a computer. Place common jobs in the OSI layers and name the matching TCP/IP idea.

What a network is for

A network lets computers share data and devices. Every exchange has five ingredients. The **sender** creates the message. The **receiver** is the intended destination. The **medium** is the path, which may be a cable or a radio link. The **message** is the data. The **protocol** is the agreed set of rules: the order of the bits, what an address looks like, and what to do if a piece is lost. Without a shared protocol the bits arrive and still cannot be understood.

Scalability is the ability to handle more users or more data without a redesign. **Reliability** is the ability to keep working when a part fails. **Availability** is the share of time the service is actually usable. A lab network that stops completely when one cable is unplugged is reliable only while every cable is perfect.

Topologies



Four topologies. The gold dot in the star is the switch or hub.

Topology	How it is joined	Strength	Weakness
Bus	One shared cable	Little cable	A break in the backbone stops everyone; collisions grow as hosts are added
Star	Each host to a central switch	One host cable can fail and the others continue; easy to add a host	If the switch fails, the whole star stops
Ring	Each host to two neighbours	Orderly access, historically by passing a token	One break can stop a single ring
Mesh	Many links, sometimes every host to every other	More than one path, so it survives failures	Cost and wiring grow quickly
Tree	Stars joined in a hierarchy	Matches a campus with buildings and floors	A failure near the root disconnects a large branch
Hybrid	A mixture	Each part can use a suitable shape	The design must be documented or nobody can repair it

Table. In the lab, 'star' almost always means a switch in the middle, not a passive hub.

On a bus, hosts share the medium. Ethernet's older shared-cable version used **CSMA**: a host listens, and it sends only if the medium is idle. If two hosts send together, both detect the collision and try again after a short random wait. A ring can use **token passing**: only the host holding the token may send, so there is no collision. A modern switched star gives each host its own cable to the switch, which is why it scales further than a single shared cable.

OSI and TCP/IP

The **OSI model** has seven layers. It is a teaching and design reference. Data is passed down the layers at the sender, each layer adding its own header, and passed up the layers at the receiver. You are not expected to memorise header

formats. You are expected to know which layer has which job.

7 Application	HTTP, DNS, SMTP
6 Presentation	Formatting, encryption
5 Session	Dialog control
4 Transport	TCP, UDP
3 Network	IP, routing
2 Data link	Frames, MAC
1 Physical	Bits on the medium

OSI is a reference model. TCP/IP is the stack the Internet actually runs.

The Internet runs the **TCP/IP** family, which is usually described in four layers. The link layer moves frames on one network. The internet layer, using **IP**, addresses hosts and routes packets between networks. The transport layer, using **TCP** or **UDP**, moves data between programs. TCP checks delivery and keeps order. UDP does not, and it is used when a late packet is useless, such as a live voice call. The application layer holds **HTTP**, **HTTPS**, **DNS**, and **SMTP**.

REMEMBER

OSI layer 3 thinks about addresses across networks. OSI layer 4 thinks about the conversation between two programs. IP is layer 3. TCP is layer 4. HTTP is layer 7, and it travels inside TCP.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which topology keeps working for the other hosts when one host's own cable is unplugged, but stops completely if the central device fails?
 - A. Bus
 - B. Star
 - C. A single ring
 - D. A full mesh with no central device
2. Compare star and ring on scalability and on what happens when one link fails.
3. Name the OSI layer responsible for routing between networks, and name the TCP/IP protocol that does that job.

ANSWERS

1. B. Each host has its own link to the centre. The centre is the shared risk.
2. A star grows by adding a port and a cable; one host cable can fail and the other hosts stay connected, but the switch is a single point of failure. A single ring does not need a central switch, but one broken link can stop the whole ring, and adding a host means breaking the ring to insert it.
3. The network layer, layer 3. The protocol is IP.

Lecture 1.6 Cybersecurity and encryption

OUTCOMES

Explain why data needs protection and name the common threats at the level of their effect. Contrast symmetric encryption, asymmetric encryption, and hashing.

What is being protected

Cybersecurity is the work of keeping systems and data available, correct, and private. Three words organise the subject. **Confidentiality** means the wrong people cannot read the data. **Integrity** means the data is not changed without detection. **Availability** means the rightful user can reach the service. A lock on a spreadsheet protects confidentiality. A checksum that fails when a file is edited protects integrity. A backup protects availability.

You should recognise the following by what they do, not by how they are constructed. **Malware** is software written to harm or misuse a system. A **virus** attaches itself to a file and spreads when that file runs. A **worm** spreads across a network by itself. **Ransomware** locks files and demands payment. **Spyware** watches what you do. **Phishing** is a message that pretends to be from a college, a bank, or a friend so that you will reveal a password or open a harmful file. A **DDoS** flood tries to make a service so busy that real users cannot reach it. A **firewall** allows or blocks traffic according to rules. **Authentication** checks who you are. **Authorization** checks what you are allowed to do after that.

Encryption

Plaintext is the readable message. **Ciphertext** is the scrambled form. **Encryption** turns plaintext into ciphertext with a key. **Decryption** reverses it. Anyone who can see the network may see the ciphertext. Without the key they should not be able to recover the plaintext. HTTPS uses encryption so that a password typed into a college portal is not readable on the café Wi-Fi.

Method	Keys	Strength	Limit
Symmetric	The same secret key encrypts and decrypts	Fast, suitable for large files. AES is the usual example.	The sender and receiver must both already have the secret. Sending that secret on an open network defeats the purpose.
Asymmetric	A public key encrypts; the matching private key decrypts	The public key can be shared openly. RSA is the usual example.	Slower. Used to agree a symmetric key, or to sign a message, more often than to encrypt a whole video.
Hashing	No key that reverses the result	Produces a short fingerprint. Used to store passwords and to detect a changed file.	It is not encryption. You cannot 'decrypt' a hash back into the original text.

Table. Hashing is grouped here because exam questions mix it with encryption. It is a different tool.

Two-factor authentication (2FA) asks for two different kinds of evidence, such as a password and a code sent to a phone, or a password and a fingerprint. A stolen password alone is then not enough. This is a control you use. It is not a second password of the same kind written on the same piece of paper.

EXAM TIP

When a question says 'contrast', use the same headings for both sides: number of keys, speed, and the problem each one solves. A list of product names does not contrast anything.

REMEMBER

Protect the account you actually have: a long password used nowhere else, 2FA where the service offers it, and a habit of opening the bank or college site by typing the address yourself rather than following a link in a message.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which control is designed to stop unwanted network connections according to a rule, rather than to scramble a message?
 - A. Symmetric encryption
 - B. A firewall
 - C. A hash
 - D. Asymmetric encryption
2. Contrast symmetric and asymmetric encryption on key use, speed, and the key-sharing problem. One sentence is enough for each point.
3. Why is storing a hash of a password safer than storing the password itself? Why can the hash not be used as the password?

ANSWERS

1. B. A firewall filters traffic. Encryption and hashing change or fingerprint data; they do not decide which connection may open.
2. Symmetric encryption uses one shared secret and is fast, but both sides must obtain that secret safely. Asymmetric encryption uses a public key and a private key, is slower, and lets the sender encrypt with a key that was published openly.
3. If the file of hashes is stolen, the original passwords are not in it, because a hash is not meant to be reversed. The hash cannot be typed in place of the password: the login system hashes what the user types and compares hashes, so the attacker would still need an input that produces the same hash.

Practical work

Do these in the lab or with Logisim and a sheet of paper. Keep the circuit and the truth table together in your practical journal.

- 1 In Logisim, build AND, OR, NOT, NAND, and XOR. Toggle every input combination and copy the truth table into your journal.
- 2 Build the circuit for $F = AB' + C'$ and check it against the K-map in Lecture 1.3.
- 3 Draw a star and a bus topology for a 6-computer lab. Mark the single point whose failure stops the whole network.
- 4 Write a one-page feasibility note for a college attendance system: technical, economic, operational, and legal.

Unit 2 Computational Thinking and Algorithms

Computational thinking is the habit of shaping a messy problem until a computer can help. You break the problem apart, ignore what does not matter, notice what repeats, and then write steps that a person or a program can follow without guessing. This unit applies that habit to searching and sorting, which appear in almost every larger program.

OUTCOMES

By the end of this unit you should be able to: Use decomposition, pattern recognition, and abstraction on a familiar problem. Write and trace pseudocode, including a decision and a loop. Trace linear search, binary search, bubble sort, and insertion sort, and say when each is appropriate.

Lecture 2.1 Decomposition, patterns, and abstraction

OUTCOMES

Define the four parts of computational thinking and apply each to one problem. Say which details an abstraction has deliberately dropped.

Four habits

Decomposition splits a problem into smaller problems that can be solved one at a time. A college result system is not one blob of work. It is 'read the marks', 'check that each mark is between 0 and 100', 'compute the total', 'decide the grade', and 'print the sheet'.

Pattern recognition looks for a step that repeats, or a new problem that is an old problem in disguise. Checking one mark and checking the next mark are the same step. A library fine and a late-fee on a bus card are both 'money owed grows with the number of late days'.

Abstraction keeps what the solution needs and sets aside the rest. A map of Karachi used for bus routes does not draw every tree. The trees are real. They are not useful for that question. A student record used for a result sheet needs the roll number and the marks. It does not need the student's favourite colour. Dropping the colour is an abstraction, not a loss of data from the world.

Algorithm design writes the remaining steps so clearly that two careful people produce the same result. The four habits are a cycle. You often abstract, then notice that the abstract problem is one you have already solved, then decompose what is still new.

EXAMPLE

Problem: decide whether a student may sit a paper. Decomposition: identify the student, read the attendance, read the fee status, apply the rule. Pattern: every student is judged by the same rule. Abstraction: the rule needs attendance percentage and fee clearance, not the student's address. Algorithm: if attendance is at least 75 and the fee is clear, print 'allowed', otherwise print the reason that failed.

REMEMBER

Abstraction is not vagueness. You should be able to name what you kept and what you dropped. If you cannot name the dropped detail, you have not made an abstraction; you have simply forgotten something.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Removing the colour of the classroom walls from a program that only computes average marks is an example of
 - A. decomposition
 - B. abstraction
 - C. a syntax error
 - D. binary search
2. A canteen wants a program that totals a student's lunch bill for the week. Apply decomposition and abstraction in two or three sentences.
3. Give one pattern that appears both in checking a list of marks for values above 80 and in checking a list of temperatures for values above 40.

ANSWERS

1. B. The colour is a real detail that the chosen problem does not use.
2. Decomposition: for each day, read the items, look up each price, add them, then add the five days. Abstraction: the program needs item names and prices, not the student's seat in the classroom or the colour of the tray.
3. Both walk through a list and count, or collect, the items that are greater than a fixed limit. The same loop solves both once the list and the limit are given.

Lecture 2.2 Algorithms, pseudocode, and flowcharts**OUTCOMES**

State the properties of a good algorithm. Write pseudocode that uses a decision and a loop, and trace it.

What makes an algorithm

An **algorithm** is a finite sequence of precise steps that takes input and produces output. Five properties are used as a checklist. It has **input**. It has **output**. Each step is **definite**, so it cannot be read in two ways. It is **finite**: it reaches the end. Each step is **effective**: a person could in principle carry it out with pencil and paper. 'Make the essay better' is not an algorithm. 'Count the words; if there are fewer than 200, stop and report the count' is closer.

Correctness means the output matches the specification for every allowed input. **Clarity** means another student can follow it. **Efficiency** means it does not do obviously wasted work. At this grade, efficiency is judged by counting the steps on a larger input, not by a formal proof. An algorithm can be clear and still be slow. A later lecture compares search methods on that basis.

Pseudocode

Pseudocode is structured English. It is not Python and it is not a paragraph. Use a verb at the start of a step. Indent the body of a decision or a loop. Name variables with words. Do not hide a loop inside the sentence 'and so on'.

EXAMPLE

Sum of the even numbers from 2 up to N, where N is a positive even input.

```
Set sum to 0
Set i to 2
While i is less than or equal to N
  Add i to sum
  Increase i by 2
Print sum
```

Trace for N = 6. Start sum = 0, i = 2. Then sum = 2, i = 4. Then sum = 6, i = 6. Then sum = 12, i = 8. The condition fails. Output is 12.

The same shape produces the product of the odd numbers from 1 to 30 if you change three things: start the result at 1 rather than 0, because a product that starts at 0 stays 0; start i at 1; and multiply instead of add. The loop structure stays. That is pattern recognition applied to your own pseudocode.

Flowcharts

A flowchart draws the same control. An oval is start or stop. A parallelogram is input or output. A rectangle is a process. A diamond is a decision, and it must have two labelled exits. Arrows do not cross if you can avoid it. A flowchart that never returns to a decision cannot express a loop. When you trace, write the values of the variables beside each arrow you follow. Tracing is how you find a loop that never ends, which violates finiteness.

EXAM TIP

If the question gives pseudocode and asks you to modify it, change as little as possible and say what you changed. Rewriting the whole algorithm from scratch often drops a step the marker was looking for.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which description fails the definition of an algorithm?
 - A. A finite list of definite steps that prints the larger of two numbers
 - B. The instruction 'improve the paragraph until it feels excellent'
 - C. A loop that adds 1 to a counter until the counter reaches 10
 - D. A recipe that stops when the water boils
2. The even-sum pseudocode in this lecture is run with $N = 4$. Show the value of sum and i after each pass of the loop, and state the output.
3. Rewrite that pseudocode so it prints the product of the odd numbers from 1 to 9.

ANSWERS

1. B. 'Feels excellent' is not definite, and there is no clear end.
2. Start sum = 0, $i = 2$. After pass 1: sum = 2, $i = 4$. After pass 2: sum = 6, $i = 6$. Then i is greater than 4, so the loop stops. Output is 6.
3. Set product to 1. Set i to 1. While i is less than or equal to 9, multiply product by i and then increase i by 2. Print product. The result is $1 \times 3 \times 5 \times 7 \times 9 = 945$.

Lecture 2.3 Linear search and binary search**OUTCOMES**

Trace both searches and count the comparisons. Choose the appropriate search for sorted and unsorted data.

Linear search

Linear search starts at the first item and compares the target with each item until it finds a match or runs out of items. It does not care whether the list is sorted. If the target is absent, every item is examined. If the target is in the last position, every item is examined as well. For a list of n items the longest run does about n comparisons. That is what people mean when they say the time grows in proportion to n .

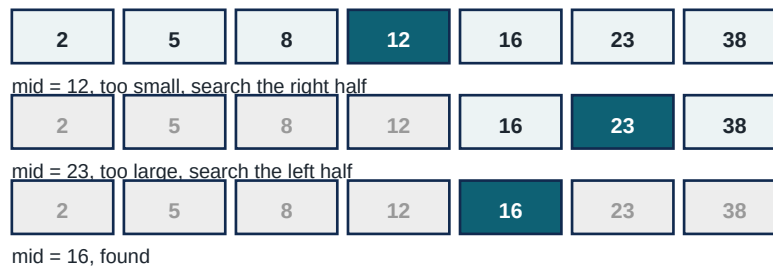
EXAMPLE

List: 5, 1, 4, 2. Target: 4.
Compare index 0 (5): no.
Compare index 1 (1): no.
Compare index 2 (4): yes. Stop.
Three comparisons. The same search for 9 examines all four items and reports that 9 is absent.

Binary search

Binary search is allowed only on a list that is already sorted. It looks at the middle item. If that item is the target, it stops. If the target is smaller, the target cannot be in the right half, so that half is discarded. If the target is larger, the left half is discarded. Each step halves the remaining list. For 1,000 sorted items, linear search may need 1,000 comparisons, while binary search needs about 10, because 2 to the power 10 is 1,024. Halving is the whole advantage. On an unsorted list the decision 'it must be in the right half' is false, so binary search must not be used.

Binary search for 16. Low, mid, and high move until mid holds the target.



Grey cells are outside the current search window. The teal cell is the middle.

Keep three markers. **Low** is the first index still in play. **High** is the last. **Mid** is the whole-number middle, $(\text{low} + \text{high}) / 2$, ignoring the fraction. When the middle is too small, set low to $\text{mid} + 1$. When it is too large, set high to $\text{mid} - 1$. When low passes high, the target is absent. Forgetting the $+ 1$ or the $- 1$ makes the same middle repeat forever, and the algorithm is no longer finite.

Question to ask	Linear search	Binary search
Must the data be sorted?	No	Yes
Comparisons in the worst case, roughly	n	$\log_2 n$
Useful when	The list is short or unsorted	The list is long and already sorted
What goes wrong if you choose badly	A long sorted list is scanned from the start	An unsorted list gives a wrong 'not found'

Table. $\log_2 n$ means 'how many times can you halve n before you reach 1'.

EXAM TIP

If the question lists scenarios such as 'small unsorted set' and 'large sorted set', name the search and give the one-line reason. The reason is the mark.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Binary search is inappropriate when
 - the list is sorted and very long
 - the list is unsorted
 - the target is the middle item
 - the list contains numbers
- Trace binary search for 5 in 2, 5, 8, 12, 16, 23, 38. Give low, mid, high, and the middle value at each step.
- A teacher has 30 unsorted scripts and wants the script of one roll number. Which search do you use, and what would you have to do before the other search became legal?

ANSWERS

- B. The half that is discarded is chosen by comparing with the order. Without order, that choice is meaningless.
- Indexes 0 to 6. low 0, high 6, mid 3, value 12. 5 is smaller, so high becomes 2. low 0, high 2, mid 1, value 5. Found.
- Linear search, because the pile is unsorted. Binary search becomes legal only after the scripts are sorted by roll number.

Lecture 2.4 Bubble sort and insertion sort

OUTCOMES

Show every pass of bubble sort and every insertion of insertion sort. Explain why both are correct and why neither is the method used for huge lists.

Bubble sort

Bubble sort walks along the list comparing each pair of neighbours. If a pair is out of order, the two values are swapped. After one full walk, the largest value has moved to the end, like a bubble rising. The next walk ignores that final position. After the second walk, the next largest value is in place. For n items there are at most $n - 1$ walks. You may stop early if a walk makes no swaps, because the list is already sorted.

Bubble sort on [5, 1, 4, 2]. Each pass walks left to right and swaps any pair that is out of order.

Start	5	1	4	2
After pass 1	1	4	2	5
After pass 2	1	2	4	5
After pass 3	1	2	4	5

The largest unsorted value settles at the right after each pass.

On [5, 1, 4, 2] the first pass swaps (5, 1), then (5, 4), then (5, 2), and finishes as [1, 4, 2, 5]. The second pass swaps (4, 2) and finishes as [1, 2, 4, 5]. The third pass makes no swap. The list is sorted. Each pass does less work than the one before it only because the settled tail is not revisited. The method is still slow on a long list that is in reverse order, because almost every pair swaps.

Insertion sort

Insertion sort keeps a sorted portion at the left. The portion starts as the first item alone, which is trivially sorted. Each new item is taken from the unsorted side and inserted into the correct place in the sorted portion, shifting larger items one step right to make a gap.

Step	Sorted portion	Item being inserted	List afterwards
Start	5	—	5 1 4 2
Insert 1	5	1 moves before 5	1 5 4 2
Insert 4	1 5	4 fits between 1 and 5	1 4 5 2
Insert 2	1 4 5	2 fits between 1 and 4	1 2 4 5

Table. The vertical bar is only a teaching mark. It is not stored.

Insertion sort is also about n -squared comparisons in the worst case, when every new item belongs at the very front. It does very little work when the list is already nearly sorted, because each new item is already close to its place. Bubble sort can also stop early on a sorted list, but only after a pass that checks the neighbours. For the short lists in an examination trace, either method is acceptable if the question names it. Do not mix the steps. A bubble-sort answer that 'inserts' a value is a different algorithm.

REMEMBER

Sorting is not searching. A sort rearranges every item. A search looks for one item. Binary search is often used after a sort, and the sort is only worth paying for if you will search many times.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. After one full pass of bubble sort toward the right, which value is guaranteed to be in its final position?
 - A. The smallest
 - B. The largest
 - C. The middle
 - D. None of them
2. Show every pass of bubble sort on 3, 1, 2. Stop when a pass makes no swap, and say whether you needed that pass.
3. Trace insertion sort on 4, 2, 3. Show the list after each insertion.

ANSWERS

1. B. Each swap moves the larger neighbour right, so the largest value ends the pass at the end.
2. Pass 1: compare 3 and 1, swap to 1, 3, 2; compare 3 and 2, swap to 1, 2, 3. Pass 2: 1 and 2 are in order, 2 and 3 are in order, no swap. Pass 2 is the check that proves the list is sorted. The list was already sorted at the end of pass 1.
3. Start with sorted portion [4]. Insert 2: [2, 4, 3]. Insert 3: 3 fits between 2 and 4, so the list is [2, 3, 4].

Practical work

Trace on paper first. Then type the same steps in Python in Unit 3 and check that the paper and the program agree.

- 1 Write pseudocode that reads 10 marks and prints how many are 50 or above. Trace it with a list you invent.
- 2 Trace linear search and binary search for the value 23 in the sorted list 2, 5, 8, 12, 16, 23, 38. Count the comparisons.
- 3 Show every pass of bubble sort on 9, 4, 6, 1.
- 4 Show every insertion of insertion sort on the same four numbers, and say which sort did less moving on this particular list.

Unit 3 Programming Fundamentals

The programming language in this course is Python. A program is a precise way of writing the algorithms from Unit 2 so that the computer, rather than you, carries out the steps. This unit covers the editor, values and types, decisions and loops, lists, functions, a small drawing library, and the habit of finding your own mistakes.

OUTCOMES

By the end of this unit you should be able to: Write and run a Python program that uses variables, decisions, loops, lists, and functions. Use the turtle library and one library function you did not write. Identify a syntax error, a runtime error, and a logic error, and dry-run code to find the last of these.

Lecture 3.1 Programs, Python, and the editor

OUTCOMES

Explain what a program is and why Python is used in this course. Name the parts of an IDE you will actually touch.

From an algorithm to a program

A **program** is an algorithm written in a language a computer can follow. The language is strict. A missing colon or a wrong indent is not a small slip of style; the program will not run. Python is the language chosen for Higher Secondary in the current curriculum because the programs are short, the errors are readable, and the same language can later talk to files, drawings, and data.

Python is a high-level language. You write `total = total + mark` rather than the machine's own instructions. An interpreter reads your file and carries the steps out. You do not produce a separate executable in the way a C++ course used to ask. You run the source file.

The editor

An **IDE**, an integrated development environment, is the place where you write, run, and look for errors. Thonny is a good classroom choice. VS Code and the Python extension are another. An online editor such as a classroom Replit works when the lab has no local install. Whichever you use, learn five things and ignore the rest until you need them.

- The editor pane, where the text of the program lives.
- The Run command. Learn its shortcut.
- The shell or console, where input is typed and print output appears.
- The file name. Yours should end in `.py`, for example `marks.py`.
- The error panel. Read the last line of the message first. It names the kind of error. The line above it names the line number.

Save before you run. A common lab failure is to edit one file and run a different file. The title bar should show the name you think you are running.

REMEMBER

The first program in any language is allowed to be small. `print("Karachi")` is a complete program. It takes no input, it produces output, and it stops. That already satisfies the definition from Unit 2.

```
print("Board of Intermediate Education Karachi")
print("Computer Science, Class XI")
```

Output

```
Board of Intermediate Education Karachi
Computer Science, Class XI
```

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which task belongs to the IDE rather than to the Python language itself?
 - A. Adding two numbers
 - B. Deciding which block of an if statement runs
 - C. Giving you a button that runs the file you are editing
 - D. Storing a value in a variable
2. Give two reasons this course uses Python rather than asking you to write machine instructions directly.
3. Name three parts of the screen you should be able to point to in the practical exam, and say what each is for.

ANSWERS

1. C. Run, the editor, and the error list are tools around the language. The language itself defines variables and decisions.
2. Python is high level, so a step such as adding a mark reads like the algorithm. The interpreter also reports errors in terms of your source line, which a string of machine codes would not.
3. The editor holds the program. The Run command starts it. The shell shows printed output and accepts input.

Lecture 3.2 Values, types, operators, and input

OUTCOMES

Use int, float, str, and bool, and convert input into a number. Evaluate an expression using Python's order of operations.

Values and names

A **variable** is a name attached to a value. The name is written on the left of `=`. The value is worked out on the right. `mark = 18` means 'from now, mark refers to 18', not 'mark is algebraically equal to 18'. A later line `mark = 20` moves the name. The old 18 is simply no longer named.

Every value has a **type**. `int` is a whole number. `float` is a number with a fractional part. `str` is text, always written in quotes. `bool` is `True` or `False`. The type decides which operations are legal. You can add two ints. You cannot add an int and a str, because the interpreter will not guess whether you meant arithmetic or joining text.

```
x = ["apple", "banana", "mango"]
print(type(x))
x = "Hello World"
print(type(x))
x = 20.5
print(type(x))
```

Output

```
<class 'list'>
<class 'str'>
<class 'float'>
```

Operators

Arithmetic uses `+`, `-`, `*`, and `/`. Division with `/` always produces a float, so `7 / 2` is 3.5. `//` is integer division, which discards the fraction: `7 // 2` is 3. `%` is the remainder: `7 % 2` is 1. `**` is power: `3 ** 2` is 9. Comparison uses `==`, `!=`, `<`, `>`, `<=`, and `>=`, and produces a bool. Do not write `=` when you mean to compare. One equals sign stores a value.

Powers happen before multiplication and division. Multiplication, integer division, and remainder happen before addition and subtraction, and they happen from left to right. Parentheses happen first of all.

```
print((2 + 3 ** 2 + 4 // 2 * 5 % 3 - 1) * 2)
```

Output

```
22
```

The steps inside that expression are: `3 ** 2` is 9; `4 // 2` is 2; `2 * 5` is 10; `10 % 3` is 1; then `2 + 9 + 1 - 1` is 11; then `11 * 2` is 22. Write those steps in an examination even if you also know the final number. The working is the answer.

Input

`input` always returns text, even when the user types digits. If you are going to do arithmetic, convert it. `int(input("Mark: "))` reads a line and makes an int. `float(...)` is the conversion when a fractional mark is allowed. A string of letters passed to `int` stops the program. That is a runtime error, and Lecture 3.7 returns to it.

```
mark_text = "18"
mark = int(mark_text)
print(mark + 2)
```

Output

```
20
```

EXAM TIP

If a program prints a number glued to text, convert with `str` or use a comma in `print`. `print("Total", total)` puts a space between them. `print("Total" + total)` is an error when `total` is an `int`.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What is the value of `7 // 2`?
A. 3.5
B. 3
C. 4
D. 1
2. A student writes `answer = input("Age: ")` and then `print(answer + 1)`. The user types 16. What happens, and what is the repair?
3. Evaluate `2 + 3 ** 2` and, separately, `(2 + 3) ** 2`. Show why they differ.

ANSWERS

1. B. Integer division discards the fraction. The remainder operator, not `//`, would give 1.
2. `input` returns the text "16", and adding 1 to text is a `TypeError`. The repair is `answer = int(input("Age: "))`.
3. `2 + 9 = 11`, because the power happens first. `(2 + 3) 2 = 5 2 = 25`, because the parentheses happen first.

Lecture 3.3 Decisions and repetition**OUTCOMES**

Write `if`, `elif`, and `else`, and choose between `for` and `while`. Predict the output of a small loop, including `range`.

One decision

An `if` runs its block only when the condition is true. The block is the indented lines. The colon is required. `elif` tries another condition when earlier ones were false. `else` runs when every condition was false. Conditions are checked from top to bottom and they stop at the first true one, so the order of the tests is part of the meaning.

```
def classify(letter):
    letter = letter.lower()
    if letter in "aeiou":
        return "vowel"
    return "consonant"

for ch in ("A", "b", "e"):
    print(ch, classify(ch))
```

Output

```
A vowel
b consonant
e vowel
```

A mark scheme with overlapping tests must be ordered. Test `mark >= 80` before `mark >= 50`. If you test `mark >= 50` first, a mark of 90 satisfies it and never reaches the distinction test. That program runs. It is still wrong. That is a logic error.

for and while

`for` is the loop you use when you know the collection, or when you know how many times. `for i in range(5)` gives `i` the values 0, 1, 2, 3, and 4. The stop value is not included. `range(1, 6)` gives 1 through 5. `range(2, 11, 2)` gives 2, 4, 6, 8, 10. The third number is the step.

```
for i in range(5):
    print(i)
```

Output

```
0
1
2
3
4
```

`while` repeats as long as a condition stays true. Use it when you do not know the number of repeats in advance, for example 'keep reading marks until the user types -1'. Something inside the loop must eventually make the condition false. A `while` whose variable never changes is not finite.

```
nums = [4, 7, 10, 3, 8]
even = 0
odd = 0
i = 0
while i < len(nums):
    if nums[i] % 2 == 0:
        even = even + 1
    else:
        odd = odd + 1
    i = i + 1
print("even", even, "odd", odd)
```

Output

```
even 3 odd 2
```

The same count is shorter with `for item in nums`, because the list itself supplies the next item and you cannot forget to increase `i`. Use `while` when there is no list yet. Use `for` when there is.

REMEMBER

`break` leaves the loop immediately. `continue` skips the rest of this pass and starts the next one. You can solve every exercise in this unit without either of them. If you use `break`, say in a comment what condition you are leaving on.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What does `for i in range(5)` print on its first and last lines?
 - A. 1 and 5
 - B. 0 and 5
 - C. 0 and 4
 - D. 1 and 4
2. A program tests `if mark >= 50` before `elif mark >= 80`. A student scores 91. Which message is printed, and how do you reorder the tests?
3. Write a `while` loop that prints the even numbers from 2 to 8 inclusive. State the output lines.

ANSWERS

1. C. `range(5)` starts at 0 and stops before 5.
2. The first condition is already true, so the pass message is printed and the distinction test is skipped. Test `mark >= 80` first, then `mark >= 50`, then `else`.
3. Set `n` to 2. While `n` is less than or equal to 8, print `n` and then add 2 to `n`. The lines are 2, 4, 6, 8.

Lecture 3.4 Lists

OUTCOMES

Create, index, and loop over a list. Explain why the last legal index is one less than the length.

A list is an ordered collection

A **list** holds several values in order under one name. `marks = [18, 15, 20, 12, 16]` has length 5. The first index is 0, so the last legal index is 4, which is `len(marks) - 1`. `marks[0]` is 18. `marks[4]` is 16. `marks[5]` does not exist, and asking for it stops the program with an `IndexError`. This off-by-one is the most common list mistake in the practical.

You may mix the operations you already know with a loop. Adding up a list is the even-sum algorithm from Unit 2, with the list supplying the numbers.

```
marks = [18, 15, 20, 12, 16]
total = 0
for m in marks:
    total = total + m
average = total / len(marks)
print(total)
print(average)
```

Output

```
81
16.2
```

Building and changing a list

- `items.append(value)` adds one item at the end.
- `items[i] = value` replaces the item already at that index. It does not insert.
- `len(items)` is the number of items, not the last index.
- `value in items` is `True` when linear search would succeed. It is the search from Lecture 2.3, written as one operator.
- A slice `items[1:4]` is a new list of the items at indexes 1, 2, and 3. The stop index is excluded, just as in `range`.

An empty list is `[]`. You can start with an empty list and append inside a loop. That is how you collect only the marks that are below 10, instead of changing the original list while you are still walking through it. Changing a list's length in the middle of a `for` loop over that same list is legal and confusing. Collect into a second list.

EXAMPLE

To count how many city names in `["Karachi", "Hyderabad", "Sukkur", "Karachi"]` are equal to Karachi, set a counter to 0 and add 1 each time the item equals that string. The count is 2. The comparison is `==`, and the text must match exactly, including capital letters.

EXAM TIP

If a question shows `fruits[3]` and the list has three city names, the answer is an `IndexError`. The legal indexes are 0, 1, and 2. Count them before you say 'Lahore'.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A list has 3 items. Which index is legal for the last item?

- A. 3
- B. 2

- C. 4
- D. -0

2. What does this program print? `cities = ["Karachi", "Lahore", "Quetta"]` then a loop that prints each city together with its index, starting from 0.
3. Explain the difference between `marks.append(14)` and `marks[0] = 14` on a list that already holds five marks.

ANSWERS

1. B. Indexes start at 0, so the last of three items is index 2.
2. 0 Karachi, 1 Lahore, 2 Quetta. One way to write the loop is `for i in range(len(cities)): print(i, cities[i])`.
3. `append` adds 14 as a sixth item. The assignment replaces whatever was at index 0 and leaves the length unchanged.

Lecture 3.5 Functions

OUTCOMES

Define a function, pass arguments, and return a value. Explain the difference between a name inside a function and a name outside it.

Why a function exists

A **function** names a piece of work so you can use it more than once. It is decomposition made visible in the program. The **definition** is the `def` block. A **call** is the line that runs it. The definition does nothing until a call happens. A function that is defined and never called does not print, does not compute, and is not 'wrong' until the question asked for it to run.

The names in the parentheses of the definition are **parameters**. The values in the parentheses of the call are **arguments**. `return` sends one value back to the caller and ends the function. A function with no `return` hands back `None`, which is not the same as 0.

```
def find_max(a, b):
    if a > b:
        return a
    return b

print(find_max(12, 7))
print(find_max(3, 9))
```

Output

```
12
9
```

```
def c_to_f(celsius):
    return celsius * 9 / 5 + 32

print(c_to_f(0))
print(c_to_f(100))
```

Output

```
32.0
212.0
```

Local names

A name assigned inside a function is **local**. It exists while the call is running and it is not visible after the call returns. A name assigned outside every function is **global**. You may read a global from inside a function. Do not assign to it

from inside a function in this course. Pass the value in, and return the result out. That keeps each function testable on its own.

Reassigning a parameter does not change the caller's variable. After `def show(n): n = n + 1`, a call `show(mark)` leaves `mark` as it was. The parameter `n` received the number, and the line `n = n + 1` attached the name `n` to a new number. This is the 'pass by value' behaviour the syllabus asks you to recognise for numbers. Lists are different only because a function can change the contents of the same list, for example with `append`, without rebinding the caller's name. If the question is about numbers, say that changing the parameter does not change the caller's number.

EXAMPLE

The area of a circle and the area of a rectangle are two functions, not one function full of a menu.

`circle_area(radius)` returns `3.1416 * radius * radius`. `rectangle_area(length, width)` returns `length * width`. The main program asks which shape, reads the measurements, calls the right function, and prints the returned number. The printing stays outside the calculation. Then each calculation can be checked with one call.

REMEMBER

Indentation shows what belongs to the function. A `print` that is meant to run once, after the function, must sit at the left margin. If it is indented under `def`, it runs on every call.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A function defines `return a + b` but the program never calls the function. What is printed?
 - A. The sum
 - B. 0
 - C. Nothing from that function
 - D. An error, always
2. Write a function `passes(mark)` that returns `True` when `mark` is 50 or more, and show the call that prints the result for 47.
3. Why should `c_to_f` return the number instead of printing it?

ANSWERS

1. C. A definition records the steps. Only a call runs them.
2. `def passes(mark): return mark >= 50`. The call is `print(passes(47))`, which prints `False`.
3. A returned number can be used in a further calculation or in a test. A printed number is only visible on the screen and cannot be reused by the caller.

Lecture 3.6 Libraries and turtle graphics

OUTCOMES

Explain abstraction in the context of a library. Use `turtle` commands to draw a polygon, including lifting the pen.

A library is an abstraction

A **library** is a collection of functions someone else has tested. You call `math.sqrt(16)` without reading how the square root is computed. That is abstraction in the sense of Unit 2: you keep the job you need, 'square root of 16', and you drop the internal steps. Python's own libraries, such as `math` and `random`, come with the language. A third-party library is installed separately. In both cases the idea is the same. The call is simple. The work inside may be long.

```
import math
print(math.sqrt(16))
print(math.pi)
```

Output

```
4.0
3.141592653589793
```

`import math` makes the names available as `math.sqrt`. You do not copy the square-root code into your file. If the question says 'use a library', an import and one call are enough. Name the library in the answer.

Turtle

The turtle library gives you a pen that moves on a window. `forward(n)` moves `n` pixels in the current direction. `right(degrees)` and `left(degrees)` turn. `penup()` lifts the pen so the next move draws nothing. `pendown()` lowers it again. `circle(radius)` draws a circle. A square is four times 'forward, then right 90'. The external angle of a regular polygon with `k` sides is $360 / k$ degrees.

```
import turtle

def square(side):
    for _ in range(4):
        turtle.forward(side)
        turtle.right(90)

square(80)
turtle.penup()
turtle.forward(120)
turtle.pendown()
for _ in range(3):
    turtle.forward(80)
    turtle.right(120)
turtle.done()
```

Running that program opens a window, draws a square of side 80, moves without drawing, and draws an equilateral triangle. `turtle.done()` keeps the window open until you close it. In a written paper you are not asked to produce the window. You are asked to say which commands move, which turn, and when the pen is up.

EXAM TIP

A common written question is to complete a loop that draws a shape. Check that the turn is outside the confusion of 'how many sides'. A hexagon turns 60 degrees, six times, not 60 times.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which call lifts the pen so that the next forward does not draw?
 - A. `turtle.forward(0)`
 - B. `turtle.penup()`
 - C. `turtle.right(0)`
 - D. `turtle.done()`
2. How do you draw a regular pentagon of side 70 with turtle? State the loop.
3. In what sense is `math.sqrt` an abstraction?

ANSWERS

1. B. `penup` leaves the position free to change without a line. `done` only keeps the window open.
2. Repeat 5 times: forward 70, then right 72, because the external angle is $360 / 5 = 72$ degrees.
3. The caller asks for the square root and does not have to know the steps inside the library. The unnecessary detail has been hidden behind one name.

Lecture 3.7 Finding and fixing mistakes

OUTCOMES

Classify an error as syntax, runtime, or logic. Dry-run a function and use a print to see a value mid-program.

Three kinds of error

A **syntax error** breaks the grammar. A missing colon, a bracket that is never closed, or an indent that does not line up. The program does not start. Read the line number.

A **runtime error** is grammatical and still fails while running, on a particular value. Dividing by zero, converting 'Ali' with `int`, and reading past the end of a list are runtime errors. The names Python prints are `ZeroDivisionError`, `ValueError`, and `IndexError`. The repair is a check before the dangerous step, or a correct index.

A **logic error** runs and finishes and gives the wrong answer. An average that divides by 100 instead of by the length of the list is a logic error. No message will point at it. You find it by predicting the answer on paper for a tiny input and comparing.

Program fragment	What happens
<code>num1 = 10; num2 = "5"; num1 + num2</code>	<code>TypeError: unsupported operand type(s) for +: 'int' and 'str'</code>
<code>fruits = ["Islamabad", "Karachi", "Lahore"]; fruits[3]</code>	<code>IndexError: list index out of range</code>
<code>num1 = 10; num2 = 0; num1 / num2</code>	<code>ZeroDivisionError: division by zero</code>

Table. These three messages are worth recognising on sight. The repair is a conversion, a legal index, or a test that the divisor is not zero.

Dry running

A dry run is a trace you do yourself. Make a column for each variable. Walk one statement at a time and write the new value. Do not do the arithmetic in your head across three lines. The point of the table is that your eye, not your memory, holds the numbers.

A print in the middle of a loop is the same idea carried out by the computer. Print the index and the value you are about to use. When the program works, remove those prints. Leaving them in is not a logic error, but it is not the output the question asked for.

Code inside a function is easier to check than code written at the top of the file, because you can call the function with one input and look at one returned value. `print(c_to_f(0))` must show 32. If it shows 32, the conversion is right regardless of what the rest of the program does. That is the beginning of testing. Class XII takes the idea further with unit tests. Here, one careful call is enough.

REMEMBER

The debugger in Thonny lets you set a breakpoint, run up to that line, and look at the variables. You do not need it for the theory paper. You do need to be able to say what a breakpoint is for: it pauses the program so you can see the state, instead of guessing.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A program runs to the end and prints an average of 0.5 when every mark is 50. What kind of error is that?
 - A. Syntax error
 - B. Runtime error
 - C. Logic error
 - D. Import error

2. The list `cities` has indexes 0, 1, and 2. A student writes `print(cities[3])`. Name the error and give a legal statement that prints the last city.
3. Describe a dry run that would catch an average divided by 100 instead of by the number of marks. Use the three marks 10, 20, and 30.

ANSWERS

1. C. There is no crash and the answer is still wrong, so the steps do not match the specification.
2. `IndexError`, list index out of range. `print(cities[2])` or `print(cities[len(cities) - 1])` prints the last city.
3. The correct total is 60 and the correct average is 20. Dividing 60 by 100 gives 0.6. The paper trace shows 0.6 against the expected 20, so the divisor is the fault.

Practical work

Type every program yourself. Copying without reading teaches nothing, and a practical examiner can ask you to change one line.

- 1 Write a program that reads two numbers and prints their sum, difference, product, and quotient. Refuse the quotient when the second number is 0.
- 2 Read a mark from 0 to 100 and print a grade: 80 and above Distinction, 50 to 79 Pass, below 50 Needs improvement. Reject a mark outside 0 to 100.
- 3 Store six monthly attendance percentages in a list and print the average and the months below 75.
- 4 Write `find_max(a, b)` and a second function that uses it to find the maximum of three numbers.
- 5 Draw a square and an equilateral triangle with turtle. Lift the pen between the two shapes.

Unit 4 Data and Analysis

Data is useful only when it is organised and when the summary you draw from it is the summary you meant. This unit treats two skills that belong together in the current course: storing related facts in a database, and reading a collection of numbers without confusing a pattern with a cause.

OUTCOMES

By the end of this unit you should be able to: Explain tables, keys, and one-to-many relationships, and sketch a simple entity-relationship diagram. Choose a suitable chart, compute a mean, and distinguish correlation from causation. Describe a straight-line model $y = mx + c$ in words.

Lecture 4.1 From a flat file to a database

OUTCOMES

Explain what goes wrong when related facts are kept in one repeating table. Define table, record, field, primary key, and foreign key.

The flat file

A flat file is one table, often a spreadsheet, holding every fact the organisation could think of. It works for a single list: one row per student, and columns for name and class. It starts to lie when one student borrows many books. If you add columns `Book1`, `Book2`, and `Book3`, the fourth book has nowhere to go. If you repeat the student's name on a new row for every book, a changed spelling has to be edited in several places, and one of them will be missed. That repeated editing is the practical meaning of redundancy.

A **database management system** stores the facts in related tables and follows rules when rows are added or removed. The database is the data. The DBMS is the software, such as Access, LibreOffice Base, or MySQL. The syllabus also expects you to know that a finished database application has forms for entering data, queries for asking questions, and reports for printing.

The words

- A **table** holds one kind of thing, such as students, or books.
- A **record** is one row.
- A **field** is one column. Each field has a type: text, number, date, or yes/no.
- A **primary key** identifies one row and does not repeat. A roll number can be a primary key. A name cannot, because two students can share it.
- A **foreign key** is a field in one table that holds the primary key of another table. It is how a loan row points at one student and one book.

Referential integrity means a foreign key is not allowed to point at a row that is not there. The DBMS can refuse a loan row whose roll number matches no student. Without that rule the database will happily store a loan by a student who does not exist, and a report will show a blank name.

DEFINITION

A query is a stored question, such as 'which books are due this week'. A form is a screen for one job, such as issuing a book. A report is a layout for paper or for a PDF. All three sit on top of the tables. They are not extra copies of the data.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which field is suitable as a primary key for students?
 - A. First name
 - B. City
 - C. Roll number, if the college never reuses it
 - D. The word 'student'
2. A spreadsheet has columns Name, Book1, Book2, and Book3. Give two problems this design will hit.
3. What is referential integrity, in one situation from a library?

ANSWERS

1. C. A primary key must identify one row. Names and cities repeat.
2. A student who borrows a fourth book has no column. And most students leave Book2 and Book3 empty, while a change to the student's name does not by itself tell you that the same person appears in another sheet.
3. The database refuses a loan that names a roll number with no matching student row, so every loan points at a real student.

Lecture 4.2 Relationships and entity-relationship diagrams

OUTCOMES

Distinguish one-to-one, one-to-many, and many-to-many relationships. Sketch entities, attributes, and a relationship, and say which relationships need their own table.

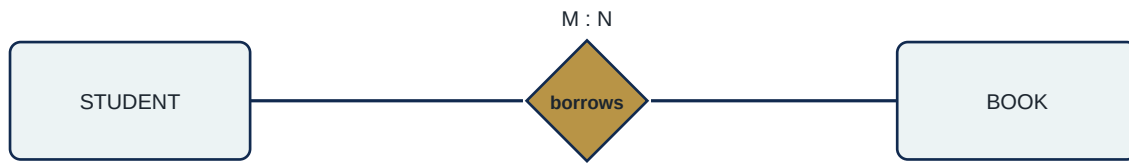
Three relationships

One-to-one. One row on the left matches at most one row on the right. A student and that student's current library card, if a student may hold only one card, is one-to-one. These are the least common, and they are sometimes just extra fields on one table.

One-to-many. One row on the left matches many rows on the right. One class has many students. One publisher has many books. The foreign key sits on the 'many' side. The student row stores the class id. The class row does not store a list of roll numbers.

Many-to-many. One student borrows many books over the year, and one book is borrowed by many students over the years. A foreign key on either side is not enough, because a single field cannot hold a list and still stay atomic.

The relationship becomes its own table. `Loan(student_id, book_id, due_date)` holds one row per borrowing. Each id is a foreign key.



Rectangles are entities. The diamond is the relationship.

A many-to-many borrow relationship needs its own table in the relational model.

Entity-relationship sketch for a college library.

How to read and draw the diagram

In the diagram used in this course, a rectangle is an entity, a diamond is a relationship, and an oval is an attribute. Write the cardinality beside the diamond: 1:1, 1:M, or M:N. You do not need drawing software in the theory paper. A tidy pencil sketch with the cardinality labelled is the answer.

Attributes belong on the entity that owns them. 'Title' is an attribute of Book. 'Due date' is an attribute of the borrowing, not of the book and not of the student, because the same book has a different due date each time it goes out. Putting the due date on the book would store only the latest loan and would forget the earlier ones.

EXAM TIP

If the question says 'draw the tables', a many-to-many relationship in the story must appear as a third table in the answer. Stopping at Student and Book loses the mark for the relationship.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. One class has many students, and each student is in one class. Where does the foreign key go?
 - A. On the class, as a list of names
 - B. On the student, storing the class id
 - C. In a third table, always
 - D. Nowhere, because this is many-to-many
2. A book can have several authors, and an author can write several books. Why is a single Author field on the Book table the wrong design?
3. Name the symbol for an entity, an attribute, and a relationship in an ER diagram.

ANSWERS

1. B. This is one-to-many. The foreign key lives on the many side.
2. The relationship is many-to-many. One text field either stores one author only, or stores a list in one cell, which breaks the rule that a field holds one value. An Author table and a BookAuthor table are required.
3. A rectangle is an entity, an oval is an attribute, and a diamond is a relationship.

Lecture 4.3 Summaries and charts

OUTCOMES

Compute a mean and a median, and choose among bar, histogram, line, and scatter. Read a chart by naming what each axis measures.

Numbers that summarise

The **mean** is the ordinary average: add the values and divide by how many there are. The marks 10, 20, and 30 have mean 20. The **median** is the middle value after sorting. For 10, 30, and 20 the sorted list is 10, 20, 30 and the median is 20. For an even count, average the two middle values. The **mode** is the value that occurs most often. A mean is pulled by one very large value. A median is not. A class mean of 70 with one mark of 0 can hide the fact that most students scored above 70. Quote the median as well when the question is about a typical student.

The **range** is the largest value minus the smallest. It is a crude measure of spread. Two classes can share a mean and differ in range. The class whose marks run from 40 to 90 is not the same situation as the class whose marks run from 68 to 72, even when both means are 70.

Which picture

Chart	Use it when	Do not use it when
Bar chart	You are comparing categories, such as pass and fail counts	The horizontal axis is a continuous measurement chopped into bins
Histogram	You want the shape of one numeric variable, such as the distribution of marks	You want to compare named categories
Pie chart	You want parts of one whole, and there are few slices	You have many slices, or the values are not parts of one total
Line graph	You are showing change over time	The horizontal axis has no order
Scatter plot	You are looking at two numeric variables together	You have only one variable

Table. A histogram answers 'how are these marks spread?'. A bar chart answers 'how many students are in each named group?'.

Read a chart in a fixed order. Name the chart. Read the axis titles and the units. Only then describe the shape: a peak, a gap, a rise, a cluster. A sentence that says 'the graph goes up' without saying what is on each axis has not been read.

EXAMPLE

Eight students score 40, 42, 55, 55, 55, 70, 90, and 91. The mode is 55. The median is the average of the 4th and 5th sorted values, which are both 55, so the median is 55. The mean is $(40+42+55+55+55+70+90+91) / 8 = 498 / 8 = 62.25$. The mean sits above the median because 90 and 91 pull it. A histogram would show a clump around 55 and a tail toward 90. A pie chart of these eight numbers would be the wrong picture, because they are not parts of one whole.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which chart is the usual choice for the distribution of a single set of examination marks?
 - Scatter plot
 - Histogram
 - Pie chart of each mark
 - Line graph with students in alphabetical order
- Find the mean and the median of 4, 8, 6. Show the sorting for the median.
- Why can the mean give a misleading picture of a 'typical' household income in a street where one household earns far more than the others?

ANSWERS

1. B. A histogram shows the shape of one numeric variable. Alphabetical order is not a time order, so a line graph would invent a trend.
2. Mean: $(4 + 8 + 6) / 3 = 6$. Sorted: 4, 6, 8. The middle value is 6, so the median is 6.
3. That one large income pulls the mean upward. Most households may earn much less than the mean. The median stays at the middle household and describes the typical street better.

Lecture 4.4 Straight-line models, correlation, and experiments**OUTCOMES**

Describe slope and intercept in a line used to predict one variable from another. Distinguish a sample from a population, and a correlation from a cause.

A line as a model

A **model** here is a simple mathematical story that links variables. The straight line $y = mx + c$ is the first model in the course. **m** is the **slope**: how much y changes when x increases by 1. **c** is the **intercept**: the value of y when x is 0. A rickshaw fare of '50 rupees plus 30 rupees per kilometre' is $\text{fare} = 30x + 50$. The slope is 30. The intercept is 50, the amount you pay even for a very short trip. Predicting a house price from its area is the same shape. Area is x . Price is y . A positive slope means a larger house is predicted to cost more. The line will not be perfect. The vertical gap from a point to the line is the error for that house.

Fitting the line by eye on a scatter plot is enough at this grade. The formal name is linear regression. You are not required to compute the least-squares formula unless a question gives you a method. You are required to say what the slope and the intercept mean in the units of the problem, and to refuse a prediction far outside the data you actually saw.

Correlation is not causation

Correlation means two variables move together. A correlation can be positive, both rising, or negative, one rising while the other falls. It can be strong or weak. **Causation** means a change in one variable produces a change in the other. Ice-cream sales and drowning incidents both rise in hot months. They are correlated. Ice cream does not cause drowning. Hot weather is a common cause of both. A scatter plot cannot, by itself, tell those stories apart.

A **population** is the whole group you care about, such as every HSSC candidate in Karachi this year. A **sample** is the group you actually measure. A **parameter** is a number about the population, often unknown. A **statistic** is the matching number computed from the sample, such as the mean age of 50 employees chosen from a company. The statistic is known. It is an estimate of the parameter, not the parameter itself.

An **experiment** assigns the condition, for example which class uses a new revision method, so that a later difference has a chance of being caused by that method. An **observational study** only records what people already do. If students who revise more also score more, the study has not proved that revising caused the scores. The students who revise more may also attend more, or sleep more. Those are other explanations. Class XII returns to this with a formal hypothesis test. Here the point is the design: if you did not assign the treatment, do not write 'caused'.

EXAM TIP

A question that says 'give a daily-life slope and intercept' wants the units. 'm is the slope' without saying 'rupees per kilometre' is incomplete.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Predicting the price of a house from its area, using a straight line, is an example of
A. clustering

- B. linear regression
 - C. sorting
 - D. many-to-many design
2. A parking fee is 100 rupees plus 50 rupees per hour. Identify the slope and the intercept, with units.
 3. Shoe size and reading age are correlated among primary-school children. Why is 'larger shoes cause better reading' an unjustified conclusion?

ANSWERS

1. B. One numeric variable is being predicted from another by a line. Clustering would group items without a predicted number.
2. The slope is 50 rupees per hour. The intercept is 100 rupees, the fee at zero hours.
3. Older children tend to have both larger shoes and higher reading ages. Age is a common cause. The correlation alone does not show that changing the shoes would change the reading.

Practical work

Use a spreadsheet for the charts and a small database, such as LibreOffice Base or Access if the lab has it, for the tables. Python is a legal way to compute the mean if a database tool is not installed.

- 1 Create Student and Book tables for a library. Give each a primary key. Decide whether 'borrows' is one-to-many or many-to-many, and add the table that decision requires.
- 2 Enter 15 invented marks. Compute the mean and the median. Plot a histogram of the marks and a bar chart of pass against needs-improvement.
- 3 Plot hours of revision against a test score for eight invented students. Draw a line by eye. Say whether the slope is positive, and write one sentence you are not allowed to conclude from the plot alone.

Unit 5 Applications and Impacts of Computing

The same ideas from systems, programs, and data show up in services you already use: a cloud drive, a sensor in a field, a model that sorts post. This unit asks what those services are made of, who is helped or harmed, and how to tell a trustworthy source from a rumour.

OUTCOMES

By the end of this unit you should be able to: Describe cloud service models and the ingredients of an Internet of Things system. Explain, at a suitable level, what a blockchain records and what artificial intelligence does not settle by itself. Judge a source of information and describe the digital divide and one assistive technology.

Lecture 5.1 Cloud, the Internet of Things, and blockchains

OUTCOMES

Compare IaaS, PaaS, and SaaS. Name the technologies that make an IoT system and a blockchain possible.

Cloud computing

Cloud computing means using computing resources over a network instead of only on the machine in front of you. The computers still exist. They stand in a data centre. You rent a share. The reason colleges care is practical: a drive that is backed up, a class site that more than one teacher can open, and a server that can grow at admission time and shrink afterwards.

SaaS	Ready-made software in the browser Google Docs, an online LMS
PaaS	A platform on which you deploy an app A host that runs your Python site
IaaS	Virtual machines, storage, networks A rented server you configure

Three cloud service models, from the most managed to the least.

SaaS, software as a service, is a finished application in the browser. **PaaS**, platform as a service, is an environment where a programmer deploys an application without managing the operating system. **IaaS**, infrastructure as a service, is a virtual machine, storage, and a network, and you install the system yourself. Moving from SaaS toward IaaS, you gain control and you take on more of the maintenance. None of the three removes the need for passwords and backups. A deleted shared folder is still deleted.

Internet of Things

An **Internet of Things** system joins small computers to the network so that a physical place can be measured and sometimes acted on. A soil sensor on a farm in Sindh, a meter on an electricity feeder, a wearable that counts steps, and a camera at a college gate are all the same shape. Something is sensed. The reading is sent. A program decides, or a person is shown the reading. The enabling changes are smaller chips, batteries that last, cheap radio links, and enough processing to filter the readings. Without the link, a sensor is only a local gauge.

IoT systems collect data about real people and real places. A smart attendance camera is not only a technical project. It is a decision about who is watched, how long the pictures are kept, and who can open them. That decision belongs in the requirements, in the same SDLC sense as any other requirement, not as a speech added at the end.

Blockchain, at this grade

A **blockchain** is a shared ledger. Records are grouped into blocks. Each new block contains a fingerprint of the block before it, so a change in an old block is visible. Many computers keep copies, which is why there is no single official copy to edit in secret. The fingerprint is a hash, from Unit 1. The copies stay in agreement through a protocol. Those two ingredients, cryptographic hashes and a network of copies, are the technologies the syllabus means when it says cryptography and distributed records enabled blockchains.

A blockchain is not automatically the right database for a college. A library loan system needs queries, corrections, and a known administrator. A blockchain is interesting when many organisations that do not fully trust one another need to share one history, for example a shipment of produce passing through several companies. 'It uses a blockchain' is not by itself a reason to trust a product.

REMEMBER

Name the service model by who manages the software. If the user opens a finished site, it is SaaS. If the programmer brings only the application, it is PaaS. If the programmer installs the operating system, it is IaaS.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Google Docs, used in the browser as a finished writing tool, is best classified as
 - IaaS
 - PaaS
 - SaaS
 - a logic gate

2. Describe an IoT design that could help a school garden, using sensor, link, and action.
3. Why does changing an old block in a blockchain get noticed?

ANSWERS

1. C. The user runs a finished application. They do not install an operating system or deploy code.
2. A moisture sensor measures the soil. A radio link sends the reading to a small server. If the reading stays below a chosen level, the system opens a valve for a short time or sends a message to the gardener.
3. Each block stores a hash of the previous block. Changing the old block changes that hash, so it no longer matches the hash stored in the next block. Copies held by other computers still have the original chain.

Lecture 5.2 Artificial intelligence and the people it affects

OUTCOMES

Distinguish a rule-based system from a system that learns from examples. Name stakeholders whose interests can conflict in one AI system.

Two ways a program can decide

A **rule-based** program follows tests a person wrote: if attendance is below 75, print a warning. The rule is visible. It does not change unless a programmer edits it. A **machine-learning** program is given examples and adjusts internal numbers so that it reproduces the examples and, the designers hope, handles new cases. A spam filter trained on old messages is the second kind. Class XII opens the training process. Here you need the contrast and a few names.

- **Supervised learning** uses examples that already have the right answer, such as 'this picture is a cat'.
- **Unsupervised learning** has no such labels. Clustering students by the pattern of their library use, without being told the 'right' group, is unsupervised.
- **Classification** predicts a category. **Regression** predicts a number, which is the line from Lecture 4.4.
- **Natural language processing** works with human language: translation, or picking the topic of a paragraph.
- **Computer vision** works with images. **Robotics** acts in the physical world.

Stakeholders

A **stakeholder** is a person or group with a real interest in the system. In a system that flags which college applications look 'strong', the stakeholders include the applicants, the admissions staff, the college, and the people who will not apply again if they believe the process is unfair. Their interests are not the same. Staff may want a shorter list. Applicants want a reason they can challenge. A model that is slightly more accurate and completely unexplained serves one of those interests and harms the other.

The syllabus asks you to see that culture and values enter the design whether or not anyone mentions them. If the training examples come from one kind of school, the model learns that kind of school. That is not a neutral fact about 'the data'. It is a decision about whose past is being copied. A responsible requirement says who can ask for a human review, what is not allowed as an input (for example a photograph when the decision should not depend on appearance), and how long the application is kept.

EXAMPLE

Positive uses are real and should be stated without advertising language. A translation tool helps a student read a paper. A vision tool can warn a driver. A timetable tool can save an office a week. The same week you should be able to name the person who is worse off if the tool is wrong, and the appeal they have.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which task is natural language processing?
 - A. Adding two integers

- B. Sorting a list of marks
 - C. Grouping an Urdu sentence by its topic
 - D. Drawing a square with turtle
2. Contrast supervised and unsupervised learning with one example each.
3. An automated short-listing tool is introduced for a scholarship. Name two stakeholders and one conflict between them.

ANSWERS

1. C. The input is human language. The other tasks are ordinary programs or graphics.
2. Supervised learning trains on examples that already have answers, such as emails marked spam or not spam. Unsupervised learning looks for groups in data that has no such labels, such as clustering shoppers by what they buy.
3. Applicants and the selection committee. The committee may want speed. Applicants want to know why they were rejected and to correct a factual mistake. A system with no explanation serves the first interest and blocks the second.

Lecture 5.3 Information, connectivity, and access

OUTCOMES

Separate a reliable source from an unreliable one using checks you can actually perform. Describe the digital divide, assistive technology, and one environmental cost of computing.

Reliable information

A source is more **reliable** when you can name the author, the date, and the evidence, and when another independent source agrees on the factual part. Official statistics, a textbook that cites its curriculum, a research paper, and a news report that quotes a document are in this family. A source is **unreliable** when there is no author, no date, no evidence, or a demand that you share it before it is deleted. A screenshot of a conversation is not a source until you know who wrote it and what was left out.

Before you use a figure in your own work, cross-check it. Bias is not only a matter of bad people. A page can be honest and still be incomplete, because the writer never looked at your city. 'I found it online' is not a citation. Write the title, the organisation, and the date you opened it.

Connectivity and its costs

Networks have increased connectivity: people study with a teacher they will not meet, families share news, and a shop sells beyond its street. The same connectivity carries harassment, fraud, and a pace of comparison that many students find exhausting. The technical fact and the human effect are both part of the answer. An answer that only praises, or only condemns, has missed half of the syllabus.

Data centres use electricity, and much of that electricity still produces carbon emissions. Phones and computers become electronic waste. These are environmental costs of the services in Lecture 5.1. They do not mean the services should vanish. They mean a design question includes energy and disposal, not only features.

The digital divide and assistive technology

The **digital divide** is the gap between people who can use digital services and people who cannot, because of money, electricity, language, skill, or disability. A college that submits assignments only through a portal has changed the rules for a student who does not have a quiet connection at home. Bridging the gap is concrete: a lab that stays open, devices to borrow, material in the languages students read, and a paper alternative when the network is down.

Assistive technology is a tool that lets someone do something a disability or age would otherwise block. A screen reader speaks the text of a page. Captions turn speech into text. A magnifier enlarges a diagram. These are not extra features for a few people at the edge of the design. A page that only works with a mouse already fails a student who uses a keyboard, and it often fails a student on a small phone as well.

EXAM TIP

Questions on this lecture are marked for a specific practice, not for the word 'awareness'. 'Cross-check the author and the date' will score. 'We should be careful online' will not.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which habit is the best example of using information responsibly?
 - A. Sharing a shocking post quickly, before it is checked
 - B. Copying a paragraph without a source
 - C. Cross-checking a claim against a named, dated source
 - D. Using any image that a search returns
2. Give two effects of the digital divide on a student, and two actions a college can take.
3. Name two assistive technologies and the barrier each one reduces.

ANSWERS

1. C. A named and dated source can be inspected. The other three hide or skip the evidence.
2. The student may miss announcements and may be unable to submit work on time. The college can keep a lab open after class and can accept the same work on paper when a student's connection fails.
3. A screen reader reduces the barrier of a page that would otherwise have to be seen. Captions reduce the barrier of speech that would otherwise have to be heard.

Practical work

These practicals are short investigations. Write half a page, not a slogan.

- 1 Pick one campus problem, such as a hot classroom or a queue at the canteen. Describe an IoT design: what is sensed, where the reading goes, and who acts on it.
- 2 Classify five digital services you used this week as IaaS, PaaS, or SaaS, and say which layer the user actually sees.
- 3 Take one news claim about technology. Find two sources. Say which is more reliable and why, using author, date, and evidence.

Unit 6 Digital Literacy and Prototyping

The last unit of Class XI is about gathering evidence and trying an idea while it is still cheap to change. Digital literacy here means more than 'using a computer'. It means designing a way to collect data and presenting what you found. Prototyping means building a small version of a solution so that real users can show you that you are wrong, early.

OUTCOMES

By the end of this unit you should be able to: Design a simple data-collection method and present the result with a suitable digital tool. Explain why a prototype is tested, and improve an idea using what the test showed.

Lecture 6.1 Searching, collecting, and presenting

OUTCOMES

Use a search that is more precise than a single word. Choose a data-collection method and match the presentation to the data.

A search with a design

A single common word returns a pile. A useful search adds constraints. Quotation marks ask for a phrase, as in "digital divide". OR keeps alternatives. A minus sign excludes a word. `site:.edu.pk` keeps pages on Pakistani educational domains. You still have to open the page and apply the reliability checks from Lecture 5.3. A clever search that lands on an undated anonymous page has not finished the job.

Collecting original data

Sometimes the information is not on the web, because it is about your own class. Then you design the collection. A **survey** asks the same questions of many people and is good for counts. An **interview** asks fewer people and follows up, and it is good for reasons. A **prototype test** watches someone try a design. A **simulation** uses a model when trying the real system would be dangerous or expensive, such as a fire drill planned on a computer rather than by starting a fire. The method has to match the question. A survey of two friends cannot support 'most students in Karachi'.

Write the question so that two students would code the answer the same way. 'Do you like the canteen and is it cheap?' is two questions, and a yes is meaningless. 'How many days last week did you buy lunch in the canteen?' has a number for an answer. Avoid a sample that is only your own friends if the claim is about the whole college. Say who was left out.

The artefact

An artefact is the thing you hand over: a slide, a one-page report, an infographic, a chart in a spreadsheet. An infographic earns its place when a reader can see the comparison without your commentary. It fails when it decorates a number that a sentence would have stated more honestly. Title the chart. Label the axes. Say how many people answered. A bar chart of fifteen classmates is a description of those fifteen, not of Pakistan.

REMEMBER

Primary data is data you collected. Secondary data is data someone else published. Both need a source line. For primary data the source is your method: who, when, and how many.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A principal wants to know why students avoid the new library timings. Which method fits best?
 - A. A simulation of the building
 - B. An interview or a small set of open questions
 - C. Bubble sort
 - D. A pie chart of the provinces
2. Rewrite the search 'computers bad for health' so that it is more likely to find serious sources. Mention two changes.
3. You surveyed 15 classmates. What must the caption of your chart say so that you do not overclaim?

ANSWERS

1. B. The question asks for reasons. An interview can follow an unexpected answer. A chart of unrelated categories cannot.
2. Search for a precise phrase such as "screen time" health, and restrict the site or look for a named organisation. Then check the author and the date of whatever opens.
3. It must say that the figures are from 15 classmates, name the question, and not present the result as a fact about all students or all of Karachi.

Lecture 6.2 Prototyping an idea**OUTCOMES**

Explain what a prototype is for. Describe a test and one change made because of the test.

A prototype is a question you can hold

A **prototype** is an early, incomplete version of a product, built so that you can learn. It is not a promise of the finished system. A paper prototype is a stack of drawn screens. A slide prototype clicks from one drawing to the next. A spreadsheet can be a prototype of a result sheet. The medium does not matter. What matters is that a real user can attempt a real task before you have spent a term programming the wrong screens.

Prototyping is cheap insurance inside the SDLC. It sits in design. The team learns that students look for the exam date on the first screen, not under a menu called Utilities. Changing a drawing costs an eraser. Changing the same decision after the database is built costs a week. This is why Agile teams prototype inside every cycle, and why even a Waterfall building project uses models and drawings before concrete.

How to test

- 1 Write the task in the user's words. 'Find tomorrow's Computer Science room.' Not 'explore the app'.
- 2 Give the prototype to someone who did not design it. If you explain every button, you are testing your explanation, not the screens.
- 3 Watch. Write down where they hesitate. Do not correct them during the task.
- 4 Change one thing that blocked the task. Test again with another person.

Tools you may mention in an answer include paper and pencil, presentation slides, a spreadsheet, and a design tool such as Figma if the lab has it. The marker is not awarding marks for the brand. The marks are for the task, the observation, and the change.

EXAMPLE

A prototype of a lost-and-found board has a button labelled 'Submit'. Two students tap it expecting to search. They wanted 'Search' on the first screen and 'Report a found item' as a second action. The change is to rename the button and to put the search box above it. That sentence, with the observation attached, is a complete answer.

EXAM TIP

Class XII will ask for a minimum viable product, which is a step beyond a prototype: something a first group of users can actually use. Do not use the two words as synonyms in Class XII. In Class XI, prototype is the word the questions use.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What is the main purpose of a prototype?
 - A. To avoid ever talking to users
 - B. To learn about a design while change is still cheap

- C. To replace testing
 - D. To encrypt the database
2. Give three reasons to prototype a college notice app before writing the full program.
 3. A classmate cannot find the logout control on your paper prototype. What do you record, and what is one legal change?

ANSWERS

1. B. A prototype exists to expose wrong decisions early.
2. You discover which notices students actually open. You can correct confusing labels while they are still drawings. You avoid spending the programming time on screens nobody uses.
3. Record that the user could not find logout during the task, and where they looked. Move logout to a labelled control on the first screen, then test with a different person.

Practical work

The practical outcome of this unit is a small artefact: a form plus a one-page summary, or a paper prototype plus three notes from users.

- 1 Write three questions for a survey about how classmates travel to college. Say who you will ask, how many, and what would make the sample misleading.
- 2 Collect at least fifteen answers. Summarise them in a spreadsheet and present one chart with a title and axis labels.
- 3 Sketch, on paper, three screens of an app that solves a campus problem. Ask two classmates to complete a task using only the paper. Write what they could not do, and change one screen.